

13-YARA Rules - YARA mean one!

Introduction



Cuando McSkidy desapareció, reinaba el caos y la incertidumbre en The Best Festival Company (TBFC). Sin embargo, incluso en su desaparición, McSkidy intentaba ayudar al equipo azul de TBFC. Siguiendo el ejemplo del proceso de comunicación de crisis, McSkidy envió lo que parecen ser varias imágenes al equipo azul desde una ubicación anónima. Estas imágenes parecían estar relacionadas con los preparativos de Pascua, pero contenían un mensaje enviado por McSkidy. El proceso de comunicación de crisis describe que un mensaje puede enviarse a través de una carpeta de imágenes con mensajes ocultos que pueden decodificarse si se conoce la palabra clave. El equipo azul debe crear una regla YARA que se ejecute en el directorio que contiene las imágenes. La regla YARA debe activarse con una palabra clave seguida de una palabra clave. Tras extraer todas las palabras clave en orden ascendente, el equipo azul podrá decodificar el mensaje.

- Objetivos de aprendizaje
- Comprender el concepto básico de YARA.
 - Aprender cuándo y por qué necesitamos usar reglas YARA.
 - Explorar los diferentes tipos de reglas YARA.
 - Aprender a escribir reglas YARA. Detecte prácticamente indicadores maliciosos utilizando YARA.

En la máquina virtual adjunta, la carpeta enviada por McSkidy se ha descargado en /home/ubuntu/Downloads/easter. Usaremos esta carpeta para ejecutar nuestras reglas de YARA.

Answer the questions below

Let's go!

No answer needed

Yara Rules

Resumen de YARA

En esta etapa, McSkidy, el defensor principal, le ha confiado una misión que implica el uso de las reglas de YARA. Antes de entrar en acción, analicemos con más detalle qué es YARA, cómo funciona y por qué es una herramienta tan valiosa en la lucha de TBFC para proteger los SOC-mas.

YARA es una herramienta diseñada para identificar y clasificar malware mediante la búsqueda de patrones únicos, las huellas digitales dejadas por los atacantes. Imagínela como el cuaderno de un detective para ciberdefensas: en lugar de buscar huellas, YARA escanea código, archivos y memoria en busca de rastros sutiles que revelen la identidad de una amenaza.



Dentro de TBFC, YARA actúa como un guardián silencioso, escaneando los sistemas y descubriendo los rastros más sutiles de actividad maliciosa que otros podrían pasar por alto. Ahora, mientras la nieve se acumula sobre Wareville y la red SOC-mas se tambalea bajo la amenaza, es su turno de usar esta herramienta y restablecer el equilibrio.

Por qué es importante YARA y cuándo usarla

Quizás se pregunte por qué McSkidy eligió esta herramienta para esta misión. En el mundo de los ciberataques, los defensores se enfrentan a un flujo interminable de alertas, archivos sospechosos y fragmentos de red anómalos. No todas las amenazas destacan; algunas se esconden a simple vista, camufladas como documentos o scripts inofensivos. Ahí es donde YARA destaca.

YARA ofrece a los defensores la capacidad de detectar malware por su comportamiento y patrones, no solo por su nombre. YARA le permite definir sus propias reglas, lo que le proporciona una visión propia de lo que constituye un comportamiento "malicioso". Y no está solo: muchas reglas de YARA ya han sido escritas por defensores de otros reinos que alguna vez se enfrentaron a amenazas similares. Puede usar, adaptar y mejorar estas reglas compartidas para fortalecer las defensas de TBFC y proteger conjuntamente los SOC-mas. Para los defensores de los SOC-mas, esto significa una detección más rápida, búsquedas más inteligentes y menos amenazas que pasan desapercibidas.

¿En qué situaciones podrían los defensores confiar en esta herramienta?

- **Análisis post-incidente:** cuando el equipo de seguridad necesita verificar si aún existen rastros de malware encontrados en un host comprometido en otras partes del entorno.
- **Búsqueda de amenazas:** búsqueda en sistemas y endpoints en busca de indicios de familias de malware conocidas o relacionadas.
- **Análisis basados en inteligencia:** aplicación de reglas YARA compartidas de otros defensores o reinos para detectar nuevos indicadores de compromiso.
- **Análisis de memoria:** examen de procesos activos en un volcado de memoria en busca de fragmentos de código malicioso.

Los ejemplos anteriores muestran usos comunes de YARA, pero los defensores suelen encontrar formas aún más creativas de aplicarlo durante las investigaciones y búsquedas.

Valores de YARA

En la lucha constante por proteger los centros de operaciones de seguridad (SOC), el tiempo y la precisión son fundamentales. Ahí es donde YARA demuestra su verdadero valor: proporciona a los defensores claridad en el caos. Con YARA, no tiene que esperar una actualización de antivirus ni una alerta de terceros. Tiene el poder de crear sus propias reglas, detectar nuevas variantes y actuar antes de que la amenaza se propague. Para los defensores de TBFC, YARA ofrece varias ventajas clave:

- **Velocidad:** escanea rápidamente grandes conjuntos de archivos o sistemas para identificar archivos sospechosos.
- **Flexibilidad:** detecta todo, desde cadenas de texto hasta patrones binarios y lógica compleja.
- **Control:** permite a los analistas definir con precisión qué consideran malicioso.
- **Compatibilidad:** las reglas pueden ser reutilizadas y mejoradas por otros defensores en diferentes reinos.
- **Visibilidad:** ayuda a conectar pistas dispersas para obtener una visión clara del ataque.

En resumen, YARA permite a los defensores pasar de la monitorización pasiva a la búsqueda activa, convirtiendo la información en acción antes de que los atacantes vuelvan a atacar.

Reglas de YARA

Ahora que entiendes qué es YARA y cuándo usarla, es hora de ver cómo funciona realmente. Para descubrir las huellas digitales del malvado Rey Malhare.

Una regla de YARA se construye a partir de varios elementos clave:

- **Metadatos:** información sobre la regla en sí: quién la creó, cuándo y con qué propósito.
- **Cadenas:** las pistas que YARA busca: texto, secuencias de bytes o expresiones regulares que marcan contenido sospechoso.
- **Condiciones:** la lógica que decide cuándo se activa la regla, combinando múltiples cadenas o parámetros en una sola decisión.

Así es como se ve en la práctica:

```
rule TBFC_KingMalhare_Trace
{
    meta:
        author = "Defender of SOC-mas"
        description = "Detects traces of King Malhare's malware"
        date = "2025-10-10"
    strings:
        $s1 = "rundll32.exe" fullword ascii
        $s2 = "msvcrt.dll" fullword wide
        $url1 = /http:\\\\.\\.*malhare.* / nocase
    condition:
        any of them
}
```

En este ejemplo, puede ver que la sección meta contiene campos como autor, descripción y fecha. Estos campos no son obligatorios, pero son muy recomendables. Le ayudan a usted y a otros defensores a comprender de qué se trata la regla, quién la creó y cuándo se escribió. A medida que su colección de reglas de YARA crece, tener metadatos claros le ahorrará tiempo; sin ellos, encontrar o actualizar la regla correcta puede convertirse rápidamente en un desafío. A continuación, vemos la sección de cadenas. Aquí es donde se definen las pistas, el texto, las secuencias de bytes o las expresiones regulares que YARA debe buscar. Finalmente, la sección de condiciones le indica a YARA cuándo marcar una coincidencia, según la lógica que defina; en nuestro caso, cualquiera de las cadenas.

Juntas, estas tres partes forman la base de cada regla de YARA. Analicemos con más detalle las cadenas y las condiciones para comprender cómo funcionan en la práctica.

Strings

Como se mencionó anteriormente, las cadenas son las pistas que YARA busca al escanear archivos, memoria u otras fuentes de datos. Representan las firmas de actividad maliciosa en fragmentos de texto, bytes o patrones que pueden revelar la presencia del código del Rey Malhare. En YARA, existen tres tipos principales de cadenas, cada una con su propia finalidad. Analicemos cada una y veamos cómo pueden ayudar a defender el reino de TBFC.

Text strings

Las cadenas de texto son el tipo más simple y común en las reglas de YARA. Representan palabras o fragmentos cortos de texto que pueden aparecer en un archivo, script o memoria. De forma predeterminada, YARA trata las cadenas de texto como ASCII y distingue entre mayúsculas y minúsculas, pero se puede modificar su comportamiento mediante modificadores especiales: pequeñas palabras clave que se añaden después de la definición de la cadena. El siguiente ejemplo muestra una regla sencilla que busca la palabra "Navidad" dentro de un archivo:

```
rule TBFC_KingMalhare_Trace
{
    strings:
        $TBFC_string = "Christmas"

    condition:
        $TBFC_string
}
```

A veces, atacantes como King Malhare intentan ocultar su código modificando la apariencia del texto dentro de un archivo, mediante codificación, trucos con mayúsculas y minúsculas o incluso cifrado. YARA ayuda a los defensores a contrarrestar estos métodos de ofuscación con potentes modificadores que amplían las capacidades de las cadenas de texto:

Cadenas que no distinguen entre mayúsculas y minúsculas (nocase)

Por defecto, YARA busca el texto exactamente como está escrito. Añadir el modificador nocase hace que la búsqueda ignore las mayúsculas y minúsculas, por lo que "Christmas", "CHRISTMAS" o "christmas" generarán el mismo resultado.

```
strings:
    $xmas = "Christmas" nocase
```

Cadenas de caracteres anchos - ancho, ASCII

Muchos ejecutables de Windows utilizan caracteres Unicode de dos bytes. Añadir "ancho" indica a YARA que también busque este formato, mientras que ASCII exige una búsqueda de un solo byte. Se pueden usar ambos juntos:

```
strings:
    $xmas = "Christmas" wide ascii
```

Cadenas XOR - xor

Los agentes de Malhare suelen codificar texto con XOR para ocultarlo de los escáneres. Mediante el modificador xor, YARA comprueba automáticamente todas las posibles variaciones XOR de un solo byte de una cadena, revelando lo que los atacantes intentaron ocultar.

```
strings:
    $hidden = "Malhare" xor
```

Cadenas Base64 - base64, base64wide

Algunos programas maliciosos codifican cargas útiles o comandos en Base64. Con estos modificadores, YARA decodifica el contenido y busca el patrón original, incluso cuando está oculto en formato codificado.

```
strings:
    $b64 = "SOC-mas" base64
```

Cada uno de estos modificadores hace que tu regla sea más inteligente y resistente, garantizando que incluso cuando King Malhare disfrace su código, los defensores de TBFC puedan descubrir la verdad.

Cadenas hexadecimales

A veces, el código de King Malhare no deja palabras legibles; en cambio, se oculta en bytes sin procesar en las profundidades de los ejecutables o la memoria. Es entonces cuando las cadenas hexadecimales entran en juego. Las cadenas hexadecimales permiten a YARA buscar patrones de bytes específicos, escritos en notación hexadecimal. Esto resulta útil cuando los defensores necesitan detectar fragmentos de malware, como encabezados de archivo, shellcode o firmas binarias, que no se pueden representar como texto sin formato.

```
rule TBFC_Malhare_HexDetect
{
    strings:
        $mz = { 4D 5A 90 00 } // MZ header of a Windows executable
        $hex_string = { E3 41 ?? C8 G? VB }

    condition:
        $mz and $hex_string
}
```

Cadenas de expresiones regulares

No todos los rastros del malware de King Malhare siguen un patrón fijo. A veces, su código muta; pequeños cambios en los nombres de archivo, URL o comandos dificultan su detección mediante texto plano o cadenas hexadecimales. Aquí es donde entran en juego las expresiones regulares. Las expresiones regulares permiten a los defensores escribir patrones de búsqueda flexibles que pueden coincidir con múltiples variaciones de la misma cadena maliciosa. Resulta especialmente útil para detectar URL, comandos codificados o nombres de archivo que comparten una estructura, pero difieren ligeramente cada vez.

```
rule TBFC_Malhare_RegexDetect
{
    strings:
        $url = /http:\/\/.*malhare.*/ nocase
        $cmd = /powershell.*-enc\s+[A-Za-z0-9+/=]+/ nocase

    condition:
        $url and $cmd
}
```

Las cadenas de expresiones regulares son potentes, pero deben usarse con cuidado; pueden coincidir con una amplia gama de datos y pueden ralentizar los análisis si se escriben de forma demasiado amplia.

Condiciones

Ahora que los defensores de TBFC saben cómo describir qué buscar mediante cadenas, es hora de aprender cuándo YARA debe decidir que se ha encontrado una amenaza. Esa lógica reside en la sección de condiciones, el núcleo de cada regla de YARA. La condición indica a YARA cuándo debe activarse la regla según los resultados de todas las comprobaciones de cadenas. Considérelo como el punto de decisión final, el momento en que el sistema confirma: "Sí, esto parece el código del Rey Malhare". Veamos algunos ejemplos básicos que los defensores usan en sus misiones diarias.

Coincidir con una sola cadena

La condición más simple: la regla se activa si se encuentra una cadena específica. Por ejemplo, la variable xmas.

```
condition:
    $xmas
```

Coincidir con cualquier cadena

Cuando se definen varias cadenas, la regla se puede configurar para que se active en cuanto se encuentre cualquiera de ellas:

```
condition:
    any of them
```

Este enfoque es útil para detectar indicios tempranos de vulnerabilidad; incluso una sola pista coincidente puede ser suficiente para llamar la atención.

Coincidir con todas las cadenas

Para que la regla sea más estricta, puede exigir que todas las cadenas definidas aparezcan juntas:

```
condition:
    all of them
```

Este enfoque reduce los falsos positivos; YARA solo marcará un archivo si todos los indicadores coinciden.

Combine la lógica usando: and, or, not.

Los defensores suelen necesitar más control sobre el comportamiento de las reglas. Los operadores lógicos permiten combinar varias comprobaciones en una sola condición, como si se creara una pequeña estrategia defensiva.

```
condition:
    ($s1 or $s2) and not $benign
```

Esto significa que la regla se activará si se encuentra \$s1 o \$s2, pero no \$benign. En otras palabras: detecta código sospechoso, pero ignora archivos de sistema inofensivos.

Utiliza comparaciones como: tamaño de archivo, punto de entrada o hash.

YARA también puede comprobar las propiedades de los archivos, no solo su contenido. Por ejemplo, puede detectar archivos inusualmente pequeños o grandes, un truco común utilizado por King Malhare para ocultar sus cargas útiles.

```
condition:
    any of them and (filesize < 700KB)
```

Aquí, la regla se activará solo cuando una de las cadenas coincida y el tamaño del archivo sea inferior a 700 KB. Ya hemos revisado los principales ejemplos de Condiciones y es hora de pasar a los casos prácticos donde estas reglas se aplican.

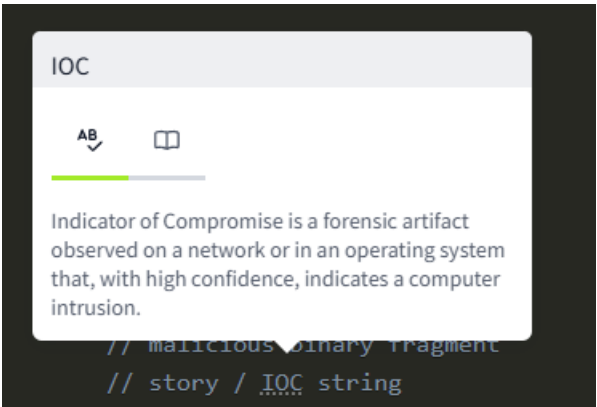
Casos de uso del estudio YARA

El reino maligno de Malhare utilizó un troyano conocido como IcedID para robar credenciales de los sistemas. Los analistas de McSkidy descubrieron que los archivos maliciosos distribuidos por Wareville compartían una firma común: el mismo encabezado MZ presente en el malware ejecutable utilizado por el Reino Oscuro. Estas muestras eran cargadores pequeños y ligeros diseñados para infiltrarse en los sistemas y posteriormente invocar cargas útiles más peligrosas. Escribamos nuestra regla YARA.

```
rule TBFC_Simple_MZ_Detect
{
    meta:
        author = "TBFC SOC L2"
        description = "IcedID Rule"
        date = "2025-10-10"
        confidence = "low"

    strings:
        $mz      = { 4D 5A }                // "MZ" header (PE file)
        $hex1    = { 48 8B ?? ?? 48 89 }    // malicious binary fragment
        $s1      = "malhare" nocase        // story / IOC string

    condition:
        all of them and filesize < 10485760 // < 10MB size
}
```



Nuestros analistas guardaron esto en un archivo llamado icedid_starter.yar y lo ejecutaron en uno de los hosts. Como resultado, podemos ver que se detectó un archivo.


```
yara -r icedid_starter.yar C:\
icedid_starter  C:\Users\WarevilleElf\AppData\Roaming\TBFC_Presents\malhare_gift_loader.exe
```

Podemos usar el comando `man yara` para averiguar qué indicadores podrían ser útiles en nuestro escenario, y encontramos lo siguiente:

- `-r` : Permite a YARA escanear directorios recursivamente y seguir enlaces simbólicos.
- `-s` : Imprime las cadenas encontradas dentro de los archivos que coinciden con la regla.

En resumen, este comando YARA escanea recursivamente la unidad C:\ usando el archivo de reglas icedid_starter.yar y, opcionalmente, puede mostrar las cadenas coincidentes al usar `-s` .

A continuación, una tarea interesante: deberás crear tu propia regla YARA. ¡Mucha suerte!

Parte práctica de YARA

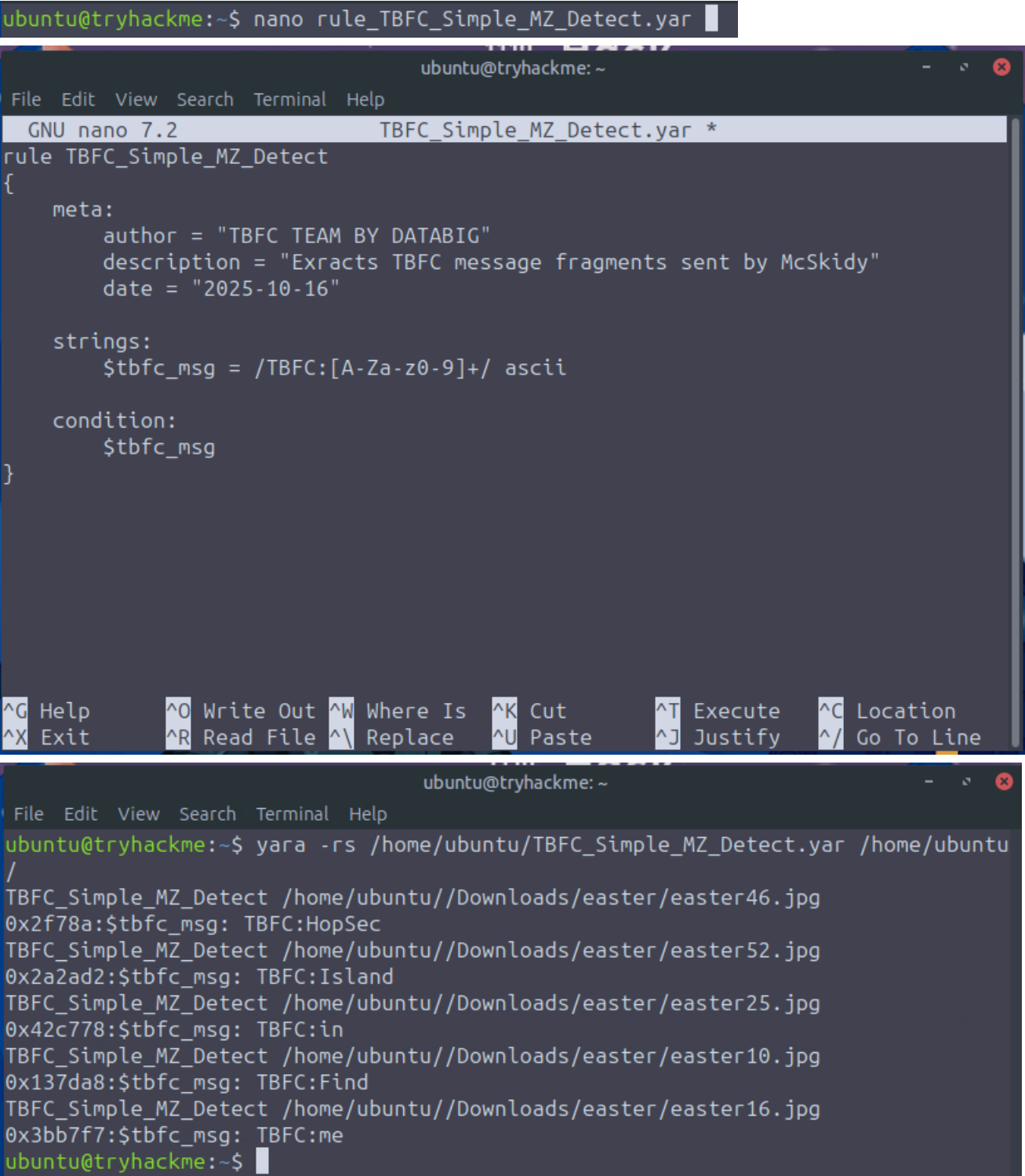
¡Es hora de completar la tarea práctica! El equipo azul debe buscar la palabra clave TBFC: seguida de una palabra clave alfanumérica ASCII en el directorio /home/ubuntu/Downloads/easter para extraer el mensaje enviado por McSkidy. ¿Puedes ayudarnos a decodificar el mensaje enviado por McSkidy?

Answer the questions below

How many images contain the string TBFC?

Respuesta: 5

creamos la regla yara para buscar el mensaje oculto y ejecutamos la regla



What regex would you use to match a string that begins with `TBFC:` followed by one or more alphanumeric ASCII characters?

Respuesta: `/TBFC:[A-Za-z0-9]+/`

What is the message sent by McSkidy?

Respuesta: Find me in HopSec Island

con la salida de la primera pregunta, organizamos las palabras en el orden numerico correcto para descifrar el mensaje oculto 10-16-25-46-52

```
ubuntu@tryhackme: ~  
File Edit View Search Terminal Help  
ubuntu@tryhackme:~$ yara -rs /home/ubuntu/TBFC_Simple_MZ_Detect.yar /home/ubuntu/  
/TBFC_Simple_MZ_Detect /home/ubuntu//Downloads/easter/easter46.jpg  
0x2f78a:$tbfc_msg: TBFC:HopSec  
TBFC_Simple_MZ_Detect /home/ubuntu//Downloads/easter/easter52.jpg  
0x2a2ad2:$tbfc_msg: TBFC:Island  
TBFC_Simple_MZ_Detect /home/ubuntu//Downloads/easter/easter25.jpg  
0x42c778:$tbfc_msg: TBFC:in  
TBFC_Simple_MZ_Detect /home/ubuntu//Downloads/easter/easter10.jpg  
0x137da8:$tbfc_msg: TBFC:Find  
TBFC_Simple_MZ_Detect /home/ubuntu//Downloads/easter/easter16.jpg  
0x3bb7f7:$tbfc_msg: TBFC:me  
ubuntu@tryhackme:~$
```

```
ubuntu@tryhackme: ~/Downloads/easter  
File Edit View Search Terminal Help  
ubuntu@tryhackme:~$ ls  
Desktop Downloads Pictures Templates script  
Documents Music Public Videos snap  
ubuntu@tryhackme:~$ cd Downloads/  
ubuntu@tryhackme:~/Downloads$ ls  
easter  
ubuntu@tryhackme:~/Downloads$ cd easter/  
ubuntu@tryhackme:~/Downloads/easter$ ls  
easter1.jpg easter21.jpg easter33.jpg easter45.jpg easter57.jpg  
easter10.jpg easter22.jpg easter34.jpg easter46.jpg easter58.jpg  
easter11.jpg easter23.jpg easter35.jpg easter47.jpg easter59.jpg  
easter12.jpg easter24.jpg easter36.jpg easter48.jpg easter6.jpg  
easter13.jpg easter25.jpg easter37.jpg easter49.jpg easter60.jpg  
easter14.jpg easter26.jpg easter38.jpg easter5.jpg easter7.jpg  
easter15.jpg easter27.jpg easter39.jpg easter50.jpg easter8.jpg  
easter16.jpg easter28.jpg easter4.jpg easter51.jpg easter9.jpg  
easter17.jpg easter29.jpg easter40.jpg easter52.jpg embeds  
easter18.jpg easter3.jpg easter41.jpg easter53.jpg  
easter19.jpg easter30.jpg easter42.jpg easter54.jpg  
easter2.jpg easter31.jpg easter43.jpg easter55.jpg  
easter20.jpg easter32.jpg easter44.jpg easter56.jpg
```